

AI DEBUGGER PRO

AI Debugger Pro: Multi-Language Execution Architecture



ABSTRACT

Developer tools often claim multi-language support by placing several command strings behind one menu. A durable execution architecture requires more: language identity, source conventions, validation stages, build artifacts, runtime semantics, diagnostics, isolation, timeouts, provenance, and verification must be represented without flattening meaningful differences. This paper analyzes the multi-language architecture of AI Debugger Pro, a local desktop prototype supporting Python, C, C++, and Java. The current implementation offers a compact registry, shared sandbox entry point, language-specific file conventions and commands, normalized execution results, common diagnosis schema, and unified human-approval workflow. Python receives pre-execution AST validation and exception-local capture; C and C++ receive compile-then-run pipelines; Java receives `javac` plus JVM execution. Docker images and commands vary by language while sharing network denial, resource constraints, a read-only container root, dropped capabilities, and an ephemeral workspace. The design succeeds at presenting a coherent user workflow, but its internal contracts remain too narrow for reliable expansion. Boolean success collapses build, runtime, timeout, infrastructure, and policy states; local and container backends return different evidence; compiler diagnostics are unstructured; toolchains are tag-pinned rather than digest-pinned; and project context is collected independently of build-system semantics. The paper proposes a capability-driven adapter model, typed phase results, execution plans, artifact manifests, diagnostic normalization, verification adapters, and conformance testing. Its central conclusion is that multi-language architecture should unify lifecycle contracts while preserving language semantics. The goal is not to make every language behave like Python wearing a different file extension; it is to make every execution explainable through the same evidence model.

Keywords: multi-language execution, compiler pipeline, language adapters, sandboxing, diagnostics, developer tools, execution provenance

1. Problem statement

AI Debugger Pro exposes one interaction model across four languages: choose a language, enter source, execute, inspect failure, request diagnosis, review a proposed replacement, approve or reject, and rerun. This consistency is a product strength. Internally, however, Python is interpreted, C and C++ compile to native binaries, and Java compiles to bytecode before JVM execution. Their syntax checks, filenames, diagnostics, artifacts, startup costs, failure classes, and verification ecosystems differ.

The architectural problem is therefore two-sided. Excessive specialization produces four unrelated tools hidden behind one interface. Excessive normalization erases the evidence needed to debug correctly. A sound design standardizes the protocol—identity, phases, results, limits, and provenance—while allowing each adapter to express its native lifecycle.

2. Scope and evidence

This paper is grounded in the Phase 6 repository. Primary evidence comes from `core/languages.py`, `core/executor.py`, `core/sandbox.py`, `core/analyzer.py`, `cli.py`, GUI routing, project discovery, and tests. It describes implemented behavior separately from proposed architecture.

The scope is single-file Python, C, C++, and Java execution under the Docker-default or explicit local backend. Arbitrary build systems, package restoration, multi-module compilation, interactive programs, services, debugging protocols, and distributed projects are outside the implemented boundary.

3. Current architecture

3.1 Language registry

`core/languages.py` defines a `Language` object containing name, extension, runner, and optional syntax checker. `SUPPORTED_LANGUAGES` registers four entries. The registry drives GUI choices and provides shared `run_code` and `check_syntax` functions. Unknown languages fail with an explicit message.

The registry is useful but only partly authoritative. The CLI maintains a separate extension-to-language map, while `core/project.py` maintains another. Container images, filenames, and commands live in `core/sandbox.py`. Adding a language therefore requires synchronized edits across several modules. This is manageable at four languages but invites configuration drift as the set grows.

3.2 Shared execution boundary

All registered languages eventually enter `run_sandboxed`, which validates language support, resolves backend choice, creates an `ExecutionResult`, and selects Docker or local execution. The result stores success, output, backend, optional return code, timeout status, and optional trace frames. `diagnostic_text` appends structured Python trace data to ordinary output for downstream diagnosis.

3.3 Language-specific paths

Language	Source name in container	Precheck	Build phase	Run phase	Enhanced failure context
Python	<code>main.py</code>	AST parse in UI path	None	Python 3.12 interpreter	Exception frames and bounded locals
C	<code>main.c</code>	No separate checker	<code>gcc main.c -o /tmp/app</code>	Native <code>/tmp/app</code>	Compiler/runtime stderr
C++	<code>main.cpp</code>	No separate checker	<code>g++ main.cpp -o /tmp/app</code>	Native <code>/tmp/app</code>	Compiler/runtime stderr
Java	<code>Main.java</code>	No separate checker	<code>javac -d /tmp Main.java</code>	<code>java -cp /tmp Main</code>	Compiler/JVM stderr

The fixed `Main.java` convention means submitted Java code must be compatible with that filename. C and C++ share an image and helper but differ by compiler and extension. Python alone receives continuous syntax validation and trace enrichment.

4. Execution lifecycle

The observable lifecycle is:

- 1 Resolve language from the GUI selection or CLI file extension.
- 2 Optionally perform a language-specific syntax check.
- 3 Select Docker by default or local execution by explicit request.
- 4 Create a temporary workspace and canonical source filename.
- 5 Add the Python trace harness when requested.
- 6 Construct the language command and execute with a timeout.
- 7 Capture stdout, stderr, return code, timeout, backend, and Python trace marker.
- 8 Normalize the result for the interface or CLI.
- 9 On failure, supply source and diagnostic text to the common AI diagnosis path.
- 10 Route a proposed full-source replacement through the common approval and rerun flow.

This architecture separates execution from AI reasoning: the model does not choose the runtime command or directly execute code. That boundary makes language support testable without requiring the model and keeps deterministic evidence upstream of probabilistic interpretation.

5. Isolation across toolchains

The Docker command applies the same baseline controls to every language: no network, 256 MiB memory, 0.5 CPU, 64-process limit, read-only container root, all capabilities dropped, no-new-privileges, a 64 MiB noexec/nosuid tmpfs, and a temporary mounted workspace. Only `PATH` is forwarded to the Docker client process.

Uniform controls simplify reasoning but do not have uniform effects. Compilers often require more memory, temporary storage, and startup time than an interpreter. Java has JVM initialization and heap behavior. C/C++ binaries may terminate by signal. Python trace capture adds another process-level wrapper. A fixed global timeout and fixed resource profile can therefore create language-biased failure rates.

Container images are selected by mutable tags: `python:3.12-alpine`, `gcc:14`, and `eclipse-temurin:21-jdk-alpine`. These convey intended versions but do not identify exact image contents. Reproducible evidence requires image digests plus captured compiler/interpreter versions.

6. Result normalization

The current `ExecutionResult` is intentionally small. Its strength is interoperability: GUI and CLI can treat every execution similarly. Its weakness is information loss.

Current field	Value	Missing distinction
success	Boolean	Validation, build, launch, runtime, test, and policy outcomes
output	Single string	stdout versus stderr, compiler versus program, truncation metadata
backend	String	Execution profile, image digest, toolchain version, command policy
return_code	Optional integer	Build code versus runtime code versus signal
timed_out	Boolean	Which phase timed out and whether descendants were terminated
trace	Python frames	Cross-language structured diagnostic schema

A compiler error, program assertion failure, missing Docker binary, unsupported language, timeout, and policy rejection should not collapse into equivalent “failure” states. They imply different explanations and different repair permissions.

7. Diagnostic asymmetry

Python diagnostics are structurally richer. The application can pre-parse source with `ast`, and the trace harness captures file, line, function, and bounded representations of locals for frames belonging to `main.py`. C, C++, and Java currently provide raw compiler or runtime text.

This asymmetry is reasonable for a prototype, but the common AI prompt may make unequal evidence look equivalent. A future adapter should emit normalized diagnostics with: phase, severity, tool, code/category, message, file, line, column, symbol, related locations, and raw text. Language-specific parsers can preserve GCC/G++ and `javac` diagnostics while the shared layer consumes one schema.

Normalization must remain lossless. Raw output should be retained within bounds because parsers can fail and toolchain formats evolve. The normalized record supplements—not replaces—the native evidence.

8. Proposed capability-driven adapter

The next architecture should replace scattered maps with one authoritative `LanguageAdapter` contract.

Adapter capability	Purpose
Identity	Stable language id, display name, source extensions, filename rules
Validation	Optional fast parser/linter before execution

Adapter capability	Purpose
Planning	Produce declared build/run phases without executing them
Environment	Image digest, toolchain constraints, resource-profile hints
Diagnostics	Parse native output into normalized records
Artifacts	Describe compiled outputs, bytecode, logs, and cleanup policy
Verification	Discover approved test adapters and expected-output checks
Context	Select relevant files without pretending to understand every build system

The adapter should generate an immutable execution plan. The sandbox executes that plan under policy; it should not contain a growing dictionary of language shell commands. This separates language knowledge from isolation policy and prevents a new adapter from silently weakening sandbox controls.

9. Typed phase model

An execution should contain ordered phase results rather than a single boolean.

Phase	Example states	Typical evidence
Resolve	supported, unsupported, ambiguous	language id and source mapping
Validate	passed, failed, skipped, unavailable	parser/linter diagnostics
Prepare	passed, failed	workspace and artifact manifest
Build	passed, failed, timed out, skipped	command id, return code, diagnostics
Run	passed, failed, signaled, timed out, not reached	stdout/stderr, exit/signal, duration
Verify	passed, failed, partial, not configured	test counts and oracle results
Policy	passed, warned, blocked	deterministic rule findings

The overall state is derived, not guessed. A Java compile failure means “build failed; run not reached.” A Python syntax failure means “validation failed.” A Docker outage means “infrastructure unavailable.” Precise states improve AI prompts, UI language, history, analytics, and repair evaluation.

10. Execution plans and artifact manifests

An execution plan should declare source inputs, canonical paths, ordered argv arrays, environment allowlist, working directory, resource profile, network policy, expected artifacts, output limits, and phase timeouts. Shell interpretation should be avoided unless a tightly controlled build adapter requires it.

An artifact manifest records what a phase produced: native executable, class files, trace data, compiler log, or test report. Artifacts receive hashes and sensitivity labels. Only declared artifacts cross phase boundaries. This makes cleanup, provenance, and verification reproducible while reducing accidental persistence.

11. Project and build-system boundaries

`core/project.py` discovers recognized source files recursively, ignores common directories, and stops at 40 files or 250,000 bytes. This is a bounded context collector, not a build-system model. It does not resolve Python packages, headers, include paths, Maven/Gradle structure, classpaths, generated code, macros, conditional compilation, or external libraries.

Future support should distinguish three modes:

- **Snippet mode:** one canonical source file, no external dependencies; current behavior.
- **Project-context mode:** bounded files inform diagnosis but are not automatically executed.
- **Declared-project mode:** a reviewed manifest identifies build root, commands, inputs, dependencies, tests, and artifacts.

The product should never infer that discovering files grants permission to build or transmit all of them.

12. Local backend parity

The local backend uses helper functions that return only (`success`, `output`) and wraps them in `ExecutionResult` without return code, timeout detail, trace data, or toolchain identity. Docker execution returns richer information. This creates evidence drift: the same UI label can rest on different facts depending on backend.

Local execution should either implement the same typed phase contract or be explicitly labeled a compatibility mode with reduced evidence. Backend parity does not mean identical isolation; it means identical result semantics and visible disclosure of security differences.

13. Extension protocol

Adding a language should require:

- 1 One adapter registration with unique id and extensions.
- 2 A canonical snippet-mode source convention.
- 3 An execution plan using `argv` arrays and declared artifacts.
- 4 A pinned runtime/build image and supported version range.
- 5 Diagnostic fixtures for syntax/build/runtime failures.
- 6 Timeout and resource-profile tests.
- 7 Success, failure, unavailable-toolchain, and malformed-output conformance tests.
- 8 Verification adapter declarations or an explicit “not supported” state.
- 9 Documentation of limitations and project-mode boundaries.
- 10 UI and CLI discovery from the same registry.

No language should require edits to the sandbox policy engine. If it does, the abstraction boundary has sprung a leak.

14. Conformance test matrix

Scenario	Python	C	C++	Java
Minimal successful program	Required	Required	Required	Required
Syntax/build error attribution	Required	Required	Required	Required
Runtime nonzero/exception	Required	Required	Required	Required
Timeout	Required	Required	Required	Required
stdout and stderr separation	Required	Required	Required	Required

Scenario	Python	C	C++	Java
Unicode source/output	Required	Required	Required	Required
Resource-limit behavior	Required	Required	Required	Required
No-network policy	Required	Required	Required	Required
Exact toolchain provenance	Required	Required	Required	Required
Fresh-workspace reproducibility	Required	Required	Required	Required

The existing tests validate Python stdout, runtime failure, timeout, Docker fail-closed behavior, local opt-in, trace parsing, and presence of selected Docker controls. Equivalent end-to-end cases are not yet present for C, C++, and Java.

15. Roadmap

M1 - consolidate identity and results

- Create one authoritative adapter registry used by GUI, CLI, project discovery, and sandbox planning.
- Introduce typed phase results with separate stdout/stderr and infrastructure states.
- Capture return codes, signals, durations, commands, toolchain versions, and truncation.

M2 - reproducible execution

- Pin images by digest and record them in every result.
- Split phase timeouts and tune resource profiles by declared capability.
- Replace embedded shell strings with argv-oriented plans where practical.
- Add artifact manifests and clean-workspace guarantees.

M3 - diagnostic and verification adapters

- Parse GCC/G++ and javac diagnostics into a common lossless schema.
- Add test adapters without conflating compilation with verification.
- Connect results to the evidence ladder defined in the repair-verification paper.

M4 - controlled project execution

- Add reviewed project manifests and dependency policies.
- Support bounded multi-file builds through explicit plans.
- Preserve context minimization and require separate consent for model transmission.

16. Acceptance criteria

A mature multi-language layer should satisfy the following conditions:

- GUI, CLI, project discovery, and sandbox use one registry.
- Every execution reports typed phases and distinguishes “not reached” from failure.
- Local and container backends expose the same evidence fields while declaring different security profiles.
- Build and runtime stdout/stderr remain separate and bounded.

- Every result identifies exact toolchain and image provenance.
- A new adapter cannot bypass global isolation policy.
- Diagnostic parsing preserves raw output.
- Zero tests cannot be reported as verification success.
- Conformance fixtures run for every registered language in CI.
- Project execution requires an explicit manifest rather than recursive-file discovery alone.

17. Limitations

This paper evaluates architecture and repository behavior; it does not benchmark cross-language performance, compiler-diagnostic quality, repair success, or container portability. The current application supports single-file examples and does not promise arbitrary ecosystem compatibility. Proposed adapters, manifests, phase schemas, and conformance suites are not implemented features.

Multi-language reproducibility is inherently bounded by operating systems, architectures, undefined behavior, locale, filesystem semantics, clocks, randomness, dependencies, and external services. The architecture can make those variables visible and controlled; it cannot wish them out of existence with a dropdown.

18. Conclusion

AI Debugger Pro has a credible multi-language spine: a shared registry, common sandbox entry point, normalized result object, language-specific toolchains, and one human-governed diagnosis-and-repair flow. That foundation is small enough to understand and strong enough to extend.

The next step is to replace boolean uniformity with contractual uniformity. Languages should share lifecycle phases, provenance, policy, and evidence semantics while preserving their native build and failure models. With capability-driven adapters, immutable plans, typed results, lossless diagnostics, artifact manifests, and conformance tests, the project can grow beyond four single-file runners without turning its core into a cabinet of command-string curiosities. The best abstraction is not the one that hides every difference. It is the one that makes every difference legible.

Appendix A. Repository evidence map

Concern	Repository evidence
Language registry and optional checker	core/languages.py
Python/C/C++/Java local helpers	core/executor.py
Docker images, filenames, commands, result object	core/sandbox.py
Independent Python analyzer	core/analyzer.py
CLI extension map and backend option	cli.py
Bounded source discovery	core/project.py
Python executor tests	tests/test_executor.py
Docker control and gating tests	tests/test_sandbox.py

Appendix B. References

- 1 T. R. Bentley. *AI Debugger Pro: Current Architecture and Human-Governed Repair Protocol*. September 2026.
- 2 T. R. Bentley. *AI Debugger Pro: Security Threat Model for Human-Governed AI-Assisted Repair*. September 2026.
- 3 T. R. Bentley. *AI Debugger Pro: Repair Verification, Evidence, and Trust*. September 2026.

- 4 Language Server Protocol Specification. Microsoft. <https://microsoft.github.io/language-server-protocol/>
- 5 OASIS. *SARIF Version 2.1.0*. 2020. <https://docs.oasis-open.org/sarif/sarif/v2.1.0/>
- 6 Reproducible Builds. <https://reproducible-builds.org/docs/>
- 7 SLSA. *Supply-chain Levels for Software Artifacts*. <https://slsa.dev/spec/>

Appendix C. Minimum adapter descriptor

The following fields define the smallest useful declaration for a future adapter. The descriptor is data, not executable authority; the sandbox policy engine validates the resulting plan before launch.

Field	Meaning	Example responsibility
id	Stable machine identity	python, c, cpp, java
display_name	User-facing label	Language selector and reports
extensions	Recognized source suffixes	CLI and project discovery
source_convention	Canonical snippet filename and entry rule	main.py or Main.java
capabilities	Validate, build, run, trace, test	UI availability without guesswork
environment	Image digest and toolchain constraints	Reproducible planning
resource_profile	Phase-specific defaults and hard maxima	Fair compiler/runtime limits
diagnostic_parser	Native-to-normalized translation	Structured build/runtime findings
artifact_rules	Declared outputs and retention	Executable, class files, reports
verification_adapters	Supported test integrations	pytest, JUnit, CTest, harness

Adapter admission should follow five rules. First, missing capabilities produce explicit unavailable states rather than optimistic defaults. Second, adapters emit plans but never launch processes directly. Third, plans cannot broaden network, mounts, environment, or privilege beyond global policy. Fourth, every native diagnostic remains recoverable from bounded raw evidence. Fifth, conformance fixtures are mandatory before registration becomes user-visible.

This contract also provides a migration path. Existing dictionaries can be generated from adapter descriptors during M1, allowing behavior to remain stable while duplicate sources of truth disappear. Typed phase results can then replace tuple and boolean boundaries incrementally. The architecture therefore does not require a dramatic rewrite; it requires a controlled transfer of authority from scattered conditionals into declared, testable contracts.